

AIA Unit 2 Notes

1. Concept of Learning & Learning Automation

1.1 What is Learning in AI?

Learning in AI refers to the ability of a system to improve its performance on tasks over time by acquiring knowledge from data and experience — without being explicitly re-programmed for every situation.

Three key things a learning system does:

- **Acquire:** Gather patterns and relationships from data or experience.
- **Generalise:** Apply what it learned to new, unseen situations.
- **Improve:** Get better at its task as it sees more data over time.

Why Learning Matters

AI systems that learn can adapt to new data, handle complex patterns that are too difficult to hand-code as rules, and continuously improve — making them far more powerful than static rule-based systems.

1.2 Learning Automation

Learning automation refers to the use of machine learning techniques to automate the process of building and improving analytical models. Instead of a human expert manually crafting rules, the system discovers those rules automatically from data.

- **Goal:** Minimise human intervention in the model-building process.
- **How:** Algorithms search through possible models and select the one that best fits the training data.
- **Examples:** AutoML systems, hyperparameter tuning, neural architecture search.

Feature	Traditional Rule-Based System	Learning Automation
Rules	Hand-crafted by domain experts	Discovered automatically from data
Adaptability	Fixed — must be manually updated	Self-updating as new data arrives
Scalability	Poor for complex or high-dimensional data	Handles large, complex datasets well
Human effort	High — experts needed for every rule	Low — system learns patterns itself

1.3 Knowledge-Based Classification

Knowledge-Based Classification is an approach where classification decisions are made using a pre-built knowledge base (a structured store of domain expert knowledge) rather than purely from statistical patterns in data.

- **Knowledge Base:** Contains facts, rules, and relationships about a domain (e.g., medical symptoms and diagnoses).
- **Inference Engine:** Applies the knowledge base to new input to produce a classification decision.
- **Example:** A medical expert system that classifies a disease based on a set of IF-THEN diagnostic rules written by doctors.

Key Distinction

Knowledge-Based Classification uses explicit human knowledge (rules). Statistical ML Classification (like Naive Bayes, SVM) learns classification boundaries from data without explicit rules.

2. Inductive & Deductive Learning

2.1 Inductive Learning

Inductive learning is a machine learning technique where a model is trained to generate predictions based on examples or observations. The model picks up knowledge from particular examples and generalises it to predict outcomes for new, unseen data. A rule or method is NOT explicitly programmed into the model.

- **Direction:** Specific examples → General rule (bottom-up)
- **Process:** Model is trained on labelled examples → spots trends and connections in the data → creates its own internal rules → applies them to new data.
- **Goal:** Make a model that can precisely anticipate the result of new, unseen instances — even when underlying patterns are complex and poorly understood.

Inductive Learning in Supervised Settings:

- A series of samples with the correct output labels are used to train the model.
- The model creates a mapping between input data and output data.
- After training, the model predicts output for fresh instances.
- **Algorithms:** Decision trees, K-nearest neighbours (KNN), neural networks.
- **Why important:** Enables models that can accurately handle new data — the foundation of most practical ML.

2.2 Deductive Learning

Deductive learning is a method where a model is built using a series of logical principles and steps. The model is specifically designed to follow a pre-defined set of rules and processes to produce predictions. Rules are explicitly programmed by domain experts.

- **Direction:** General rules → Specific conclusions (top-down)
- **Process:** Starts with a set of rules → applies those rules logically to generate predictions on new data.
- **Goal:** Make a model that can precisely follow predetermined rules and guidelines to generate reliable predictions.
- **Usage:** Rule-based systems, expert systems, knowledge-based systems — wherever rules are clearly defined by domain experts.
- **Algorithms:** Decision trees (in rule mode), rule-based systems, expert systems.

2.3 Inductive vs Deductive — Comparison

Feature	Inductive Learning	Deductive Learning
Starts from	Specific examples and data	General rules and principles

Feature	Inductive Learning	Deductive Learning
Direction	Specific → General	General → Specific
Rules	Learned automatically from data	Explicitly programmed by experts
Adaptability	Adapts as new data arrives	Fixed — rules must be updated manually
Handles uncertainty?	Yes — learns probabilistic patterns	Limited — rules are deterministic
Examples	Decision trees (ML), Neural networks	Expert systems, Rule-based systems
Analogous to	Learning by experience and practice	Learning by logic and deduction
Used in	Most modern ML / AI applications	Expert systems, formal reasoning

3. Types of Machine Learning

3.1 Supervised Learning

Supervised learning trains models on labelled data, where both input features and the desired output (label) are provided. It works like a teacher or supervisor guiding the machine — each input has the correct answer attached to it.

- **Concept:** Trains on labelled data — both inputs and correct outputs are given.
- **Goal:** Learn a mapping function that accurately predicts or classifies new, unseen data based on patterns from the training set.
- **Process:** Machine is trained on labelled examples → learns the input-output mapping → applies the mapping to new data.

Example:

A labelled dataset of animal images where each image is tagged as 'Elephant', 'Camel', or 'Cow'. After training, the model classifies new animal images it has never seen.

3.1.1 Feedback-Based Classification

In supervised learning, the model receives feedback in the form of the correct label (ground truth) for every training example. This feedback is used to compute the error and adjust the model's parameters accordingly.

- **Feedback signal:** The difference between predicted output and actual label (the loss/error).
- **How feedback is used:** The error is backpropagated through the model to update weights — this is feedback-based learning in action.
- **Result:** With each iteration, the model reduces its error and improves its predictions.

3.1.2 Regression

A supervised learning task used to predict a continuous numerical value. Establishes the relationship between a dependent variable (outcome) and one or more independent variables (features).

- **Examples:** Predicting house prices, forecasting stock values, estimating patient age from medical scans.

3.1.3 Classification

A supervised learning task that assigns data points into predefined discrete categories or classes.

- **Binary Classification:** Two classes: spam/not spam, disease/no disease.
- **Multi-class Classification:** Multiple classes: recognising handwritten digits (0-9), animal species.
- **Examples:** Spam detection, image recognition (cat/dog), medical diagnosis.

	Regression			Classification
Output type	Continuous	number	(e.g., 250,000)	Discrete category (e.g., 'Spam')

	Regression	Classification
Goal	Predict a value	Assign a label/class
Examples	House price, temperature	Email category, disease diagnosis
Algorithms	Linear Regression, SVR	Logistic Regression, SVM, Decision Tree

3.2 Unsupervised Learning

Unsupervised machine learning analyses and clusters unlabelled data to discover hidden patterns, structures, and relationships — without any prior knowledge of the desired outcomes. Unlike supervised learning, it does NOT rely on labelled data or guidance from a 'teacher'.

- **Input:** Raw, unlabelled data with no correct answers provided.
- **Goal:** Find hidden structure, patterns, or groupings within the data.
- **Challenge:** The system must determine what is 'meaningful' on its own — there is no explicit feedback signal.

Clustering

Clustering is an unsupervised technique that groups similar data points together into clusters based on their inherent similarities.

- **Concept:** Identify natural groupings within data without prior knowledge of cluster labels.
- **Example 1:** Customer segmentation: grouping customers by purchase history for targeted marketing.
- **Example 2:** Grouping stars by luminosity and temperature in astronomy.
- **Common Algorithm:** K-Means Clustering, Hierarchical Clustering, DBSCAN.

3.3 Semi-Supervised Learning

Semi-supervised learning sits between supervised and unsupervised learning. It uses a small amount of labelled data combined with a large amount of unlabelled data during training. This is practical because labelled data is expensive and time-consuming to obtain, while unlabelled data is abundant.

- **Why useful:** Labelling data requires human expert effort — semi-supervised learning leverages the vast pool of unlabelled data to improve model accuracy with minimal labelling cost.
- **How it works:** The model first learns basic patterns from the small labelled dataset, then uses the structure of the unlabelled data to refine its understanding.
- **Examples:** Web page classification (few labelled pages, millions unlabelled), medical image analysis (few expert-labelled scans, many unlabelled).
- **Techniques:** Self-training, co-training, label propagation, generative models.

Type	Labelled Data	Unlabelled Data	When to Use
Supervised	All data labelled	Not used	When labels are easy to obtain

Type	Labelled Data	Unlabelled Data	When to Use
Unsupervised	No labels	All data used	When no labels are available
Semi-Supervised	Small labelled set	Large unlabelled set	When labelling is costly/limited

3.4 Reinforcement Learning

Reinforcement Learning (RL) is a type of machine learning where an agent learns to make decisions by interacting with an environment. The agent receives rewards for good actions and penalties for bad ones, learning to maximise cumulative reward over time.

Core Components of Reinforcement Learning:

- **Agent:** The learner or decision maker (e.g., a robot, a game-playing program).
- **Environment:** Everything the agent interacts with (e.g., a game board, the physical world).
- **State (S):** A representation of the current situation of the environment.
- **Action (A):** A move the agent can take from a given state.
- **Reward (R):** Feedback from the environment after an action — positive for good moves, negative for bad.
- **Policy (π):** The strategy the agent follows — maps states to actions.
- **Value Function:** Estimates the expected cumulative reward from a given state under a policy.

Reinforcement Learning Loop:

```

Agent observes State (S)
↓
Agent chooses Action (A) based on Policy
↓
Environment transitions to new State (S')
↓
Agent receives Reward (R)
↓
Agent updates Policy to maximise future rewards
↓ (loop repeats)

```

Key Characteristics:

- **No labelled data:** The agent learns from trial-and-error interaction, not from labelled examples.
- **Delayed rewards:** An action's consequences may only become clear many steps later.
- **Exploration vs Exploitation:** Agent must balance trying new actions (exploration) vs using known good actions (exploitation).

Famous Examples:

- **AlphaGo / AlphaZero:** Defeated world champions in Go and Chess using RL.
- **Atari Games:** DeepMind's DQN learned to play Atari games from raw pixels.
- **Robotics:** Robots learning to walk, grasp objects, or navigate environments.

- **Self-driving Cars:** Agents trained to navigate roads and avoid obstacles.

RL vs Supervised Learning

Supervised learning: learns from labelled examples provided by a teacher. Reinforcement learning: learns from rewards/penalties received by interacting with an environment — no teacher, only feedback signals.

3.5 All Learning Types — Master Comparison

Learning Type	Data Needed	Feedback	Key Goal	Classic Examples
Supervised	Labelled (input + output pairs)	Direct — correct label given	Predict/classify new data	Spam filter, image classifier
Unsupervised	Unlabelled data only	None — no labels	Find hidden structure/groups	Customer segmentation, anomaly detection
Semi-Supervised	Mostly unlabelled + small labelled	Partial feedback	Leverage unlabelled data cheaply	Web search, medical imaging
Reinforcement	Agent-environment interaction	Reward/penalty signals	Maximise cumulative reward	Game playing, robot navigation
Inductive	Specific labelled examples	Correct labels	Generalise from examples to rules	Most ML algorithms (DT, ANN, KNN)
Deductive	Explicit rules from experts	Rule adherence	Apply rules to new inputs	Expert systems, rule-based AI

4. Deep Learning

4.1 What is Deep Learning?

Deep Learning is a type of machine learning and AI that trains computers to learn from large datasets in a way that simulates human cognitive processes. It mimics neural networks of the human brain, enabling computers to autonomously uncover patterns and make informed decisions from vast amounts of unstructured data.

- Deep learning models can be taught to perform classification tasks and recognise patterns in photos, text, audio, and other data types.
- It automates tasks that normally need human intelligence — describing images, transcribing audio, translating languages.
- Where human brains have millions of interconnected neurons, deep learning features neural networks constructed from multiple layers of software nodes.
- Deep learning models are trained using large sets of labelled data and neural network architectures.

Deep Learning in the AI Hierarchy

AI $\supset$ Machine Learning $\supset$ Deep Learning
Deep Learning is a specialised subset of ML that uses multi-layered neural networks. All deep learning is ML, but not all ML is deep learning.

4.2 Benefits of Deep Learning

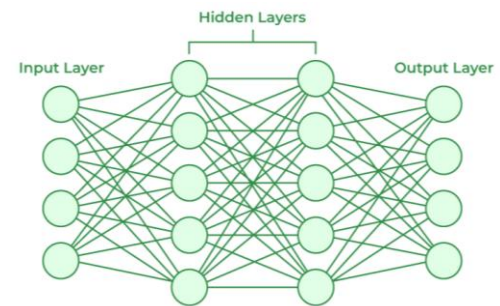
Benefit	Description
Automatic Feature Learning	Performs feature extraction automatically — does not require manual engineering of features.
Pattern Discovery	Analyses large amounts of data to uncover complex patterns in images, text, and audio.
Handles Volatile Datasets	Categorises and sorts datasets with large variations (e.g., fraud detection, transactions).
Multiple Data Types	Processes both structured and unstructured data (text, images, audio, video).
High Accuracy	Additional node layers optimise models for greater accuracy.
Outperforms Traditional ML	Needs less human intervention and handles data that traditional ML cannot process as well.
Scalability	Performance keeps improving as data volume grows — unlike traditional ML that plateaus.
Cost Effectiveness	Reduces errors significantly; the cost of inaccurate predictions often exceeds training costs.

5. Artificial Neural Networks (ANN)

5.1 Structure of an ANN

Artificial Neural Networks (ANNs) contain artificial neurons called units, arranged in a series of layers that together form the whole network. The number of layers and units per layer determines how complex the patterns the network can learn.

- **Input Layer:** Receives raw data from the outside world (e.g., pixel values, sensor readings, text tokens).
- **Hidden Layers:** One or more intermediate layers that transform input into progressively more abstract representations. The 'depth' of a deep learning network is the number of hidden layers.
- **Output Layer:** Provides the final result — a class label, a number, or a probability distribution.



5.2 Connections and Weights

In the majority of neural networks, units are interconnected from one layer to another. Each connection has a weight that determines the influence of one unit on another.

- **Weights:** Numerical values on connections. High weight = strong influence; low/negative weight = weak/inhibitory influence.
- **Biases:** Additional parameters added to each neuron — allow the activation function to shift, giving the network more flexibility.
- **Learning:** As the network trains, it adjusts weights and biases to minimise prediction error. This is how learning 'happens' in an ANN.

5.3 Activation Functions

An activation function is a mathematical function applied to the output of each neuron. It decides whether the neuron 'fires' (activates) and maps the weighted sum of inputs to an output value.

Why activation functions are essential:

- They introduce non-linearity into the network.
- Without them, the entire network — no matter how many layers — would only compute linear transformations, severely limiting what it can learn.
- Non-linearity allows the network to learn curves, edges, complex shapes, language patterns, and much more.

Activation Function	Formula / Behaviour	Key Use
Sigmoid	Output between 0 and 1 — S-shaped curve	Binary classification output layers

Activation Function	Formula / Behaviour	Key Use
Tanh	Output between -1 and 1 — zero-centred	Hidden layers (better than sigmoid in practice)
ReLU (Rectified Linear Unit)	Output = $\max(0, x)$ — zero for negatives, linear for positives	Most common hidden layer function in deep networks
Softmax	Converts outputs to probability distribution summing to 1	Multi-class classification output layers

Biological Analogy

In biological neurons, a neuron fires (sends a signal) only when the input stimulation exceeds a threshold. Activation functions in ANNs mimic this — deciding whether a neuron's input is strong enough to 'fire' and pass a signal to the next layer.

6. Backpropagation

6.1 What is Backpropagation?

Backpropagation (short for Backward Propagation of Errors) is the algorithm used to train artificial neural networks. Its goal is to reduce the difference between the model's predicted output and the actual output by adjusting the weights and biases in the network.

It works iteratively, minimising the cost (loss) function by following the error gradient. It uses the chain rule from calculus to compute gradients efficiently across all layers.

6.2 How Backpropagation Works — Step by Step

1. Forward Pass: Input data is fed through the network layer by layer, producing a predicted output.
2. Loss Calculation: The error (loss) between the predicted output and the actual (correct) label is computed using a loss function (e.g., Mean Squared Error, Cross-Entropy).
3. Backward Pass: The error is propagated backwards through the network. Using the chain rule of calculus, the gradient of the loss with respect to each weight is computed.
4. Weight Update: Each weight is updated by subtracting a fraction of its gradient: $\text{new_weight} = \text{old_weight} - \text{learning_rate} \times \text{gradient}$
5. Repeat: Steps 1–4 are repeated for each training example (or batch) across many epochs until the loss is minimised.

Core Update Rule (Gradient Descent):

$$W_{\text{new}} = W_{\text{old}} - \alpha \times \partial L / \partial W$$

Where:

W = weight

α = learning rate (how large a step to take)

$\partial L / \partial W$ = gradient of loss L with respect to weight W

6.3 Why Backpropagation is Important

Reason	Explanation
Efficient Weight Update	Computes gradient of the loss for every weight using chain rule — allows all weights to be updated in one backward pass.
Scalability	Scales well to deep networks with many layers and complex architectures — making deep learning practically feasible.
Automated Learning	The learning process becomes automated and self-correcting; the model adjusts itself to optimise performance.

► **Backpropagation Explained:** <https://www.youtube.com/watch?v=tUoUdOdTkRw>

6.4 Optimisation Algorithms

Backpropagation computes the gradients, but an optimisation algorithm decides how to use those gradients to update weights:

- **Gradient Descent:** Updates weights using the gradient computed over the entire training dataset. Slow for large datasets.
- **Stochastic Gradient Descent (SGD):** Updates weights using the gradient from one training example at a time. Faster but noisier.
- **Mini-Batch Gradient Descent:** Updates using a small batch of examples. Best of both worlds — widely used in practice.
- **Adam Optimiser:** Adaptive learning rate method — adjusts learning rate per weight. Most popular in modern deep learning.

7. Convolutional Neural Networks (CNN)

7.1 What is a CNN?

A Convolutional Neural Network (CNN) is a specialised type of deep neural network designed primarily for processing structured grid data — especially images. CNNs use convolution operations to automatically detect spatial features (edges, textures, shapes, objects) without manual feature engineering.

- **Key advantage over standard ANN:** CNNs share weights across the input (via the kernel), dramatically reducing parameters while preserving spatial relationships.
- **Applications:** Image classification, object detection, facial recognition, medical imaging, autonomous driving.

7.2 CNN Architecture & Key Components

1. Kernel (Filter)

A kernel is a small matrix of weights (e.g., 3×3 or 5×5) that slides across the input image. At each position, it performs a dot product between its weights and the corresponding patch of the input — producing a single value in the output feature map.

- **Purpose:** Detects specific features — one kernel might detect vertical edges, another detects diagonal lines, another detects curves.
- **Multiple Kernels:** Each convolutional layer typically uses many kernels in parallel, producing multiple feature maps.

Convolution Operation (simplified):					
Input patch:			Kernel:		Output value:
1	2	3	1	0	1
4	5	6	0	1	0
7	8	9	1	0	1

$(1 \times 1) + (2 \times 0) + (3 \times 1) + (4 \times 0) + (5 \times 1) + (6 \times 0) + (7 \times 1) + (8 \times 0) + (9 \times 1) = 15$

2. Stride

Stride is the step size by which the kernel moves across the input image.

- **Stride = 1:** Kernel moves one pixel at a time — output is nearly the same size as input. Captures fine-grained features.
- **Stride = 2:** Kernel jumps two pixels at a time — output is roughly half the input size. Reduces spatial dimensions.

3. Padding

Padding adds extra pixels (usually zeros) around the border of the input image before applying convolution.

- **No Padding (Valid):** Output is smaller than input — border pixels contribute less.
- **Same Padding (Zero Padding):** Adds zeros so output has the same spatial size as input. Preserves spatial dimensions and ensures border features are not lost.

4. Pooling

Pooling is a down-sampling operation that reduces the spatial dimensions (height and width) of feature maps while retaining the most important information.

- **Max Pooling:** Takes the maximum value from each region (most common). Retains the strongest feature activation.
- **Average Pooling:** Takes the average value from each region.
- **Benefits:** Reduces computation in subsequent layers, reduces overfitting, provides some spatial invariance (small shifts in the image don't change the output drastically).

5. Flattening

After the convolutional and pooling layers have extracted spatial features, the 2D feature maps are converted into a 1D vector. This flat vector is then passed to a fully-connected (dense) layer for final classification or regression.

Complete CNN Pipeline:

```
Input Image (e.g., 32×32×3 RGB)
    ↓
Convolutional Layer + ReLU Activation ← extracts local features
    ↓
Pooling Layer                        ← reduces spatial size
    ↓
[Repeat Conv + Pool layers as needed]
    ↓
Flatten                             ← 2D feature maps → 1D vector
    ↓
Fully Connected (Dense) Layer
    ↓
Output Layer (Softmax for classification)
    ↓
Class Label (e.g., 'Cat', 'Dog', 'Car')
```

► **CNN Explained (Video):** <https://www.youtube.com/watch?v=Y1qxl-Df4Lk>

7.3 ANN vs CNN — Comparison

Feature	Standard ANN	CNN
Input	Flat 1D vector	2D/3D grid (images, volumes)
Connections	Fully connected — every neuron connects to all neurons in next layer	Local connections — kernel covers small patch
Parameter count	Very high for image input	Much lower — shared kernel weights
Spatial structure	Ignored — image flattened to vector	Preserved — kernels operate on local regions

Feature	Standard ANN	CNN
Feature extraction	Manual or learned globally	Automatic hierarchical feature learning
Best for	Tabular data, simple classification	Images, video, spatial data

8. Quick Revision — Unit II at a Glance

8.1 All Topics — One-Line Summary

- **Concept of Learning:** AI system improves performance by acquiring patterns from data/experience without re-programming.
- **Learning Automation:** Automatically builds and improves models from data — minimising human expert involvement.
- **Knowledge-Based Classification:** Uses a pre-built expert knowledge base + inference engine to classify inputs.
- **Inductive Learning:** Learns general rules from specific examples (bottom-up). Foundation of most ML.
- **Deductive Learning:** Applies pre-defined rules to produce predictions (top-down). Used in expert systems.
- **Feedback-Based Classification:** Supervised learning where the error between prediction and correct label drives weight updates.
- **Supervised Learning:** Learns from labelled data. Sub-types: Regression (continuous output), Classification (discrete output).
- **Unsupervised Learning:** Finds hidden structure in unlabelled data. Key technique: Clustering.
- **Semi-Supervised Learning:** Uses small labelled + large unlabelled dataset. Reduces expensive labelling effort.
- **Reinforcement Learning:** Agent learns by interacting with environment, receiving rewards/penalties. Goal: maximise cumulative reward.
- **Deep Learning:** Multi-layered neural networks that automatically learn hierarchical features from raw data.
- **ANN:** Layers of interconnected neurons (units) with learnable weights and activation functions.
- **Backpropagation:** Training algorithm: forward pass → compute loss → backward pass (chain rule) → update weights.
- **CNN:** Specialised ANN for spatial data. Uses Kernel, Stride, Padding, Pooling, Flatten.

8.2 CNN Component Quick Reference

Component	What it Does	Key Parameter
Kernel / Filter	Slides over input, detects local features (edges, textures)	Kernel size (e.g., 3×3, 5×5)
Stride	Controls how far kernel moves each step	Stride value (1 = fine, 2 = coarse)
Padding	Adds border pixels to preserve spatial size	'Valid' (no pad) or 'Same' (zero pad)
Pooling	Down-samples feature maps, reduces computation	Pool size (e.g., 2×2), Max or Avg
Flatten	Converts 2D maps to 1D vector for dense layers	Output size = $H \times W \times \text{Channels}$

8.3 Learning Types — Rapid Fire

Type	Has Labels?	Teacher?	Feedback
Supervised	Yes — all	Yes	Correct label for every example
Unsupervised	No	No	None — discovers structure alone
Semi-Supervised	Partial (few)	Partial	Labels for small subset only
Reinforcement	No labels	No (just environment)	Reward or penalty per action

8.4 YouTube Links

- ▶ **Backpropagation:** <https://www.youtube.com/watch?v=tUoUdOdTkRw>
 - ▶ **CNN Explained:** <https://www.youtube.com/watch?v=Y1qxl-Df4Lk>
-